

1. Cadastro, login e perfis

Resumo com trechos principais dos códigos, sem exercícios. Os trechos são recortes: não representam arquivos completos. As regras descritas correspondem ao código local analisado em 09/10/2026.

Conta e senha

O sistema usa e-mail para entrar e guarda a senha de forma protegida.

Arquivo: `accounts/models.py`

```
usuario = self.model(email=email, **extra_fields)
usuario.set_password(password)
usuario.save(using=self._db)
return usuario
```

`create_user` monta a conta. `set_password` transforma a senha em hash; `save` grava a conta. `return` entrega a conta criada.

`email` e `extra_fields` fornecem os dados usados para montar `Usuario`. A variável `usuario` representa a nova conta. A senha original é usada por `set_password` para gerar o hash, que permite conferir futuros logins. Exemplo: a mesma conta criada aqui passa a ser reconhecida nas telas protegidas.

Para ler esse código: `set_password`: gera o hash da senha · `save`: grava as informações no banco · `return`: encerra a função e entrega o resultado.

Acesso ao painel

O usuário precisa entrar para abrir seu painel.

Arquivo: `accounts/views.py`

```
@login_required
def dashboard(request):
    """Mostra o painel protegido particularizado pelo perfil do usuario."""
```

`login_required` exige login. `request` contém o pedido do navegador e identifica o usuário atual. O restante da função prepara o painel.

O símbolo `@` coloca uma regra antes da função. Quando alguém abre `/dashboard/`, essa regra confere se existe uma sessão autenticada. Depois, a função usa `request.user` para identificar o perfil e preparar os blocos e avisos daquele usuário.

Para ler esse código: `request`: reúne as informações do pedido enviado pelo navegador.

Quem pode solicitar pedidos

Somente Receptor autenticado pode enviar uma solicitação.

Arquivo: `accounts/validacao_pedido.py`

```
if not getattr(solicitante, "is_authenticated", False) or solicitante.perfil !=
    Usuario.Perfil.RECEPTOR:
    raise PermissionDenied("Somente Receptor pode enviar solicitacao de pedido de sangue.")
```

A condição confere login e perfil. Se não forem permitidos, `PermissionDenied` interrompe a ação. Esconder um botão não substitui essa conferência.

`getattr` consulta `is_authenticated`; `False` é o valor usado se essa informação estiver ausente. O operador `or` significa que basta uma das situações para recusar: estar sem login ou ter outro perfil. `!=` significa diferente. A mensagem explica ao usuário por que a ação foi interrompida.

Para ler esse código: `getattr`: lê uma informação do objeto e aceita um valor padrão se ela estiver ausente · `PermissionDenied`: interrompe uma ação que a conta não pode executar · `if`: executa o bloco quando a condição é atendida · `or`: aceita que pelo menos uma das condições ligadas seja atendida · `False`: representa uma condição falsa ou desligada.

Cadastro e perfis: o fluxo completo

Como o perfil é representado

Os perfis são alternativas definidas na conta, usadas para decidir quais ações cada pessoa pode executar.

Arquivo: `accounts/models.py`

```
class Perfil(models.TextChoices):
    """Tipos que podem ser escolhidos no cadastro."""

    DOADOR = "DOADOR", "Doador"
    RECEPTOR = "RECEPTOR", "Receptor"
    HEMOCENTRO = "HEMOCENTRO", "Hemocentro"
    OBSERVADOR = "OBSERVADOR", "Observador"
    ADMINISTRADOR = "ADMINISTRADOR", "Administrador"
```

`models.TextChoices` reúne o valor que o sistema salva e o nome apresentado na tela. `DOADOR` é o valor usado nas verificações; `Doador` é o texto legível. Cada conta tem um perfil, e as views e serviços consultam esse valor.

Do formulário à conta

O formulário recebe nome, e-mail, senha, perfil e os demais dados solicitados. Os métodos `clean` conferem campos e combinações: documento apropriado, e-mail, senha e preenchimentos exigidos. Quando os dados estão válidos, a view cria a conta e registra o consentimento geral.

O salvamento usa uma transação: as gravações agrupadas precisam terminar juntas. Depois do cadastro, o Django inicia a sessão. Nas próximas páginas, `request.user` identifica a conta dessa sessão. Isso permite montar o painel e filtrar os dados do próprio usuário.

Como as permissões são aplicadas

O template escolhe os botões que aparecem. A view e o serviço conferem se a ação pode ser executada. Essas verificações se complementam: mesmo quando alguém abre uma rota diretamente, o servidor decide pelo perfil e pelas condições da operação. O acesso administrativo do Django também usa `is_staff`, `is_superuser` e permissões próprias.

Perfis: Doador faz triagem; Receptor solicita; Observador consulta; Hemocentro aprovado gerencia sua instituição; Administrador valida e modera. Visitante usa as consultas públicas.

2. Hemocentros e pedidos

Hemocentro aprovado

A instituição só publica quando seu cadastro está aprovado.

Arquivo: `accounts/validacao_hemocentro.py`

```
return (
    usuario_e_hemocentro(usuario)
    and usuario.status_validacao == Usuario.StatusValidacaoHemocentro.APROVADO
)
```

As duas condições precisam ser verdadeiras: ser Hemocentro e ter estado APROVADO. O Administrador aprova, recusa ou pede correção; o histórico guarda a decisão.

`and` significa que as duas verificações precisam ser atendidas ao mesmo tempo. `usuario_e_hemocentro` confere o tipo de conta; `status_validacao` confere a situação institucional. O resultado é `True` ou `False`, usado por outras funções para decidir se a publicação pode continuar.

Para ler esse código: `status_validacao`: guarda a situação institucional atual · `return`: encerra a função e entrega o resultado · `and`: exige que as condições ligadas sejam atendidas juntas.

Solicitar não é publicar

A solicitação começa como ENVIADA. O Hemocentro aprovado de destino analisa antes da publicação.

Arquivo: `accounts/validacao_pedido.py`

```
pedido = PedidoSangue(
    solicitante=solicitante,
    status=PedidoSangue.Status.ENVIADA,
    **dados,
)
```

`PedidoSangue` reúne o solicitante e os dados do formulário. `ENVIADA` mantém o pedido fora da lista pública. As etapas seguintes conferem os dados e salvam.

`solicitante` liga o pedido à conta que o enviou. `**dados` distribui os dados recebidos nos campos do pedido, como tipo sanguíneo e destino. O estado `ENVIADA` representa o início da análise. A decisão posterior fica registrada no histórico, junto de quem a tomou.

Publicação e aviso compatível

Um pedido só gera convocação quando está publicado e seu Hemocentro está aprovado.

Arquivo: `accounts/pedidos.py`

```
if pedido.status != PedidoSangue.Status.PUBLICADA or not
    pode_publicar_pedido(pedido.hemocentro_destino):
    return 0
doadores = doadores_aptos_para_convocacao(pedido.tipo_sanguineo).select_for_update()
```

`return 0` encerra sem aviso quando uma condição falha. Caso permitido, a mesma busca de doadores usada no estoque seleciona os candidatos. Depois são conferidos frequência e duplicidade.

A primeira condição confere o estado do pedido; a segunda confere o Hemocentro. `tipo_sanguineo` fornece o tipo necessário para a busca de candidatos. `select_for_update` solicita o bloqueio dos registros durante a transação, coordenando a conferência do limite quando mais de uma convocação acontece ao mesmo tempo.

Para ler esse código: `select_for_update`: solicita bloqueio dos registros durante a transação · `return`: encerra a função e entrega o resultado · `if`: executa o bloco quando a condição é atendida · `or`: aceita que pelo menos uma das condições ligadas seja atendida.

O Receptor acompanha suas solicitações. O Hemocentro acompanha apenas as destinadas a ele. A consulta pública lista publicados e permite filtros. O Administrador atua na moderação prevista, sem substituir a publicação institucional.

Instituições e pedidos: decisões registradas

Estado atual e histórico

Na decisão institucional, o sistema atualiza o cadastro e cria um registro separado da análise.

Arquivo: `accounts/validacao_hemocentro.py`

```
status_anterior = hemocentro_atualizado.status_validacao
hemocentro_atualizado.status_validacao = status
hemocentro_atualizado.save(
    update_fields=["status_validacao", "atualizado_em"]
)
validacao = ValidacaoHemocentro.objects.create(
    hemocentro=hemocentro_atualizado,
    admin=admin,
    status=status,
    parecer=parecer,
)
```

`status_anterior` guarda o estado antes da decisão. `status_validacao` recebe o novo estado e `save` grava os campos indicados. `ValidacaoHemocentro.objects.create` acrescenta o histórico, relacionando instituição, administrador, decisão e parecer.

Para ler esse código: `save`: grava as informações no banco · `update_fields`: indica quais campos devem ser gravados · `create`: cria e salva um novo registro · `objects`: dá acesso às operações de busca e gravação do `model` · `status_validacao`: guarda a situação institucional atual.

Como o pedido percorre o sistema

O Receptor preenche a solicitação e escolhe um Hemocentro aprovado de destino. O servidor confere título, tipo sanguíneo, urgência, cidade, descrição, identificação do solicitante e e-mail para retorno. A solicitação salva começa como ENVIADA e fica disponível para análise da instituição responsável.

Ao publicar, o sistema registra quem publicou e quando, muda o estado para PUBLICADA e prepara notificações compatíveis. Recusa e solicitação de correção também ficam no histórico. As decisões são auditadas. Possíveis pedidos semelhantes são sinalizados para ajudar a análise, sem afirmar automaticamente que são ilegítimos.

Separação dos dados

O Receptor vê suas solicitações; o Hemocentro vê as destinadas a ele; a consulta pública vê os pedidos publicados. Essa separação vem dos filtros da consulta. O identificador do pedido escolhe o registro, e as regras do servidor conferem se a conta pode agir sobre ele.

3. Triagem e resultado

Quem pode responder

A triagem pertence ao Doador e é uma orientação, não uma liberação médica.

Arquivo: `accounts/triagem_servico.py`

```
return usuario.perfil in PERFIS_COM_TRIAGEM
```

A função decide se o perfil pode responder. As telas também limitam o acesso às triagens do próprio usuário.

Essa verificação faz parte das regras de acesso à triagem. O catálogo fornece as perguntas, e o serviço cuida de iniciar, retomar, salvar respostas e concluir. Assim, as perguntas da tela e o resultado calculado ficam ligados à conta do Doador que respondeu.

Para ler esse código: `return`: encerra a função e entrega o resultado.

Versão simplificada

A versão rápida usa uma triagem extensa concluída como base.

Arquivo: `accounts/triagem_servico.py`

```
if modalidade == Triagem.Modalidade.SIMPLIFICADA:
    extensa_base = obter_extensa_base(usuario)
    if extensa_base is None:
        raise TriagemSimplificadaIndisponivel(
            "Conclua primeiro uma triagem extensa."
        )
```

Se o usuário escolher SIMPLIFICADA, o sistema procura a extensa de base. Sem essa base, interrompe o início e orienta a concluir a extensa.

modalidade indica qual versão foi escolhida. `obter_extensa_base` busca a extensa que pode servir de referência. `None` significa que nenhuma foi encontrada. Quando existe uma base, o fluxo rápido aproveita as informações permitidas e faz as perguntas necessárias para a situação atual.

Para ler esse código: `if`: executa o bloco quando a condição é atendida · `None`: representa ausência de valor.

Escolha do resultado

O motor mantém os problemas encontrados e escolhe o resultado de maior restrição.

Arquivo: `accounts/triagem_motor.py`

```
encontrados = {achado["resultado"] for achado in achados}
for resultado in PRIORIDADE_RESULTADOS:
    if resultado in encontrados:
        return resultado
return Triagem.Resultado.SEM_IMPEDIMENTO
```

achados são as condições encontradas nas respostas. O sistema percorre a ordem de prioridade e devolve a primeira presente. Sem impedimento encontrado, devolve SEM_IMPEDIMENTO.

A primeira linha reúne os resultados presentes nos achados. O `for` percorre PRIORIDADE_RESULTADOS na ordem definida pelo sistema. O primeiro resultado encontrado encerra a função com `return`. Os demais achados continuam guardados, permitindo mostrar os motivos que levaram à orientação.

Para ler esse código: PRIORIDADE_RESULTADOS: define a ordem dos resultados mais restritivos · `return`: encerra a função e entrega o resultado · `for`: percorre os itens um por um · `if`: executa o bloco quando a condição é atendida.

Os catálogos definem perguntas e regras; o formulário recebe respostas; o serviço salva, retoma e revisa; o motor calcula resultado e prazos. O histórico guarda respostas, resultado e versão da regra.

Triagem: da resposta ao histórico

Salvar ou corrigir uma resposta

Uma pergunta fica identificada por seu código dentro da triagem.

Arquivo: `accounts/triagem_servico.py`

```
RespostaTriagem.objects.update_or_create(
    triagem=registro,
    id_pergunta=id_pergunta,
    defaults={
        "codigo_resposta": codigos[0],
        "resposta_label": _rotulo_resposta(pergunta, codigos),
        "data_evento": _primeira_data(valor),
        "metadata": {
            chave: conteudo
            for chave, conteudo in valor.items()
            if chave not in {"codigos", "datas"}
        },
        "valor": valor,
        "rule_version": pergunta["regra_version"],
        "source_ref": pergunta["fonte"],
    },
)
```

`update_or_create` procura a combinação `triagem` e `id_pergunta`. Quando existe, atualiza a resposta; quando não existe, cria. `defaults` contém o que será salvo. O código guarda a alternativa, o texto, datas, valores organizados e a referência da regra.

Para ler esse código: `update_or_create`: atualiza o registro encontrado ou cria um novo · `defaults`: contém os valores usados na criação ou atualização · `objects`: dá acesso às operações de busca e gravação do model · `for`: percorre os itens um por um · `if`: executa o bloco quando a condição é atendida.

Papel de cada parte

O catálogo descreve perguntas e alternativas. O formulário monta os campos e confere o preenchimento. O serviço confirma que a pergunta pertence ao fluxo, confere os códigos escolhidos e salva a resposta. O motor lê esses dados para aplicar regras e calcular a orientação.

Na extensa, o fluxo pode incluir perguntas condicionais conforme as respostas. A simplificada usa uma extensa de base e pode abrir blocos detalhados. Algumas escolhas levam ao cancelamento da tentativa rápida e à orientação de fazer a extensa. Esses estados ajudam a preservar o histórico do que aconteceu.

Resultado e prazo

Os achados guardam as condições identificadas. O resultado principal segue a prioridade das regras; os achados continuam disponíveis para explicar os motivos. Quando há mais de um prazo, o cálculo considera a data mais distante. A versão da regra salva permite entender qual conjunto de regras produziu aquele resultado.

4. Estoque e alerta crítico

Situação do estoque

O Hemocentro aprovado registra entrada, saída ou ajuste. O sistema calcula a situação pela quantidade.

Arquivo: accounts/estoque.py

```
if quantidade_bolsas <= nivel_critico:
    return Estoque.StatusCalculado.CRITICO
if quantidade_bolsas <= nivel_minimo:
    return Estoque.StatusCalculado.BAIXO
return Estoque.StatusCalculado.ESTAVEL
```

Com mínimo 10 e crítico 5: quantidade 12 é estável, 7 é baixa e 5 é crítica. O sinal <= inclui o limite na situação indicada.

A ordem dos if é importante: o código verifica crítico antes de baixo, porque uma quantidade crítica também poderia estar abaixo do mínimo. A primeira condição atendida encerra o cálculo. Entrada soma bolsas, saída retira e ajuste define a quantidade informada; depois disso, o estado é recalculado.

Para ler esse código: return: encerra a função e entrega o resultado · if: executa o bloco quando a condição é atendida.

Disparo somente no crítico

A atualização de estoque crítico pode criar convocação; estoque baixo não gera novos avisos.

Arquivo: accounts/estoque.py

```
STATUS_DE_ESTOQUE_QUE_GERAM_ALERTA = {
    Estoque.StatusCalculado.CRITICO: Notificacao.Tipo.ESTOQUE_CRITICO,
}
```

```
if status_calculado not in STATUS_DE_ESTOQUE_QUE_GERAM_ALERTA:
    return 0
```

O mapa contém apenas CRITICO. Quando o estado não está no mapa, return 0 encerra a geração de avisos.

O dicionário liga um estado do estoque a um tipo de notificação. not in significa que o estado não aparece entre as chaves desse dicionário. Quando está crítico, o fluxo segue para buscar os doadores e preparar os avisos. A quantidade de avisos criados é devolvida para o fluxo da movimentação.

Para ler esse código: return: encerra a função e entrega o resultado · if: executa o bloco quando a condição é atendida.

Evitar excesso e repetição

Antes de criar o aviso, o sistema confere frequência e se há aviso igual ainda não lido.

Arquivo: accounts/estoque.py

```
if limite_convocacao_atingido(doador) or Notificacao.objects.filter(
    usuario=doador, estoque=estoque, tipo=tipo_notificacao, lida=False,
).exists():
    continue
```

Se uma dessas condições acontecer, continue pula esse doador. A movimentação também grava histórico e auditoria.

exists verifica se já há registro com o mesmo doador, estoque e tipo, ainda não lido. O operador or permite impedir o novo aviso por frequência ou repetição. continue atua dentro do for: pula apenas aquele candidato e permite conferir os próximos.

Para ler esse código: filter: seleciona registros pelas condições informadas · exists: responde se há algum registro correspondente · objects: dá acesso às operações de busca e gravação do model · continue: pula para o próximo item da repetição · if: executa o bloco quando a condição é atendida.

Estoque: quantidade, histórico e convocação

Registro da movimentação

Depois de salvar quantidade e situação, a função guarda o histórico e chama a geração de avisos.

Arquivo: accounts/estoque.py

```
movimentacao = EstoqueMovimentacao.objects.create(
    estoque=estoque_atual,
    usuario_resp=usuario_resp,
    tipo_movimento=tipo_movimento,
    quantidade_anterior=quantidade_anterior,
    quantidade_movimentada=quantidade_movimentada,
    quantidade_nova=quantidade_nova,
    motivo=motivo_limpo,
)
notificacoes_geradas = criar_notificacoes_para_doadores_compativeis(
    estoque=estoque_atual,
    status_calculado=status_calculado,
)
```

O histórico liga estoque e responsável e guarda quantidade anterior, mudança, quantidade nova e motivo. notificacoes_geradas recebe quantos avisos foram criados. Esse número também compõe o contexto registrado na auditoria.

Para ler esse código: create: cria e salva um novo registro · objects: dá acesso às operações de busca e gravação do model · quantidade_anterior: é o total de bolsas antes da movimentação · quantidade_nova: é o total depois da movimentação.

Como a quantidade muda

Entrada adiciona bolsas ao total; saída retira; ajuste define o novo total informado. O serviço confere tipo de movimento, quantidade, motivo e autorização. A quantidade final é usada para calcular o estado. O estado salvo alimenta a consulta do estoque e o fluxo de alerta.

Como o aviso é preparado

Quando o resultado é crítico, o sistema seleciona os doadores elegíveis e confere cada candidato. Para os aprovados nessas conferências, monta Notificacao com usuário, estoque, tipo, título, mensagem e destino. bulk_create salva os avisos preparados em lote.

A seleção e a conferência de frequência acontecem em transação, com bloqueio solicitado por doador. Isso coordena convocação de estoque e pedido quando ambos tentam usar o limite do mesmo usuário. A proteção contra aviso igual não lido também evita repetição desnecessária para o mesmo estoque.

5. Compatibilidade, aptidão e consentimento

Tipo compatível

A tabela liga o tipo solicitado aos tipos de doadores aceitos pelo sistema.

Arquivo: accounts/compatibilidade.py

```
tipo = normalizar_tipo_sanguineo(tipo_solicitado)
return COMPATIBILIDADE_RECEBIMENTO[tipo]
```

A primeira linha limpa e confere o tipo informado. A segunda busca os tipos compatíveis na tabela. Para O-, a tabela seleciona O-.

tipo_solicitado é o sangue necessário no estoque ou pedido. normalizar_tipo_sanguineo padroniza o texto e rejeita valores desconhecidos. Os colchetes consultam a tabela pela chave escolhida. Essa mesma regra alimenta a tela informativa e a seleção dos candidatos aos alertas.

Para ler esse código: return: encerra a função e entrega o resultado.

Quem pode receber convocação

Não basta ter sangue compatível: a conta precisa estar ativa, sem suspensão e com a preferência ligada.

Arquivo: accounts/compatibilidade.py

```
Usuario.objects.filter(
    perfil=Usuario.Perfil.DOADOR, is_active=True, suspensa=False,
    aceita_notificacoes_pedidos=True,
    tipo_sanguineo__in=doadores_compativeis_para(tipo_solicitado),
)
```

filter restringe os candidatos a Doadores. A consulta completa também exige a última triagem concluída apta, data de liberação permitida e consentimento aceito, não revogado e da versão configurada.

Cada linha do filter acrescenta uma exigência. is_active=True mantém contas ativas; suspensa=False exclui suspensas; aceita_notificacoes_pedidos=True exige a preferência ligada. __in significa pertencer ao conjunto de tipos compatíveis. As conferências seguintes combinam essas condições com triagem e consentimento.

Para ler esse código: filter: seleciona registros pelas condições informadas · objects: dá acesso às operações de busca e gravação do model · tipo_sanguineo__in: exige que o sangue pertença ao conjunto informado · True: representa uma condição verdadeira ou ligada · False: representa uma condição falsa ou desligada.

Limite compartilhado

Estoque e pedidos compartilham o limite padrão de um aviso em 24 horas.

Arquivo: config/settings.py

```
CONVOCACAO_INTERVALO_HORAS = 24
CONVOCACAO_LIMITE_NOTIFICACOES = 1
CONVOCACAO_VERSAO_CONSENTIMENTO = "1.0"
```

O sistema conta os avisos da janela de tempo, inclusive lidos. A preferência pode ser autorizada ou revogada pelo Doador no painel; somente a preferência ligada não substitui o consentimento registrado.

As três constantes são configurações usadas pelas funções, não dados de um único usuário. A contagem considera a janela das últimas 24 horas a partir do momento da busca. Exemplo: quem já recebeu um alerta de estoque nessa janela pode atingir o limite antes da convocação de um pedido.

Convocação: as conferências combinadas

Última triagem e consentimento

A busca prepara duas verificações ligadas ao Doador candidato.

Arquivo: accounts/compatibilidade.py

```
ultima_triagem = Triagem.objects.filter(
    usuario=OuterRef("pk"), status=Triagem.Status.CONCLUIDA,
).order_by("-finalizada_em", "-iniciada_em", "-id_triagem")
consentimento = ConsentimentoLGPD.objects.filter(
    usuario=OuterRef("pk"),
    tipo_termo=ConsentimentoLGPD.TipoTermo.NOTIFICACOES,
    versao_termo=settings.CONVOCACAO_VERSAO_CONSENTIMENTO,
    aceito=True, revogado_em__isnull=True,
)
```

OuterRef("pk") liga a busca ao usuário que está sendo analisado. A triagem é ordenada da mais recente para a mais antiga. O consentimento precisa ser do termo NOTIFICACOES, na versão configurada, aceito e sem data de revogação.

Para ler esse código: filter: seleciona registros pelas condições informadas · order_by: ordena os resultados; menos antes do campo coloca os mais recentes primeiro · OuterRef: liga a busca interna ao identificador do usuário da busca principal · objects: dá acesso às operações de busca e gravação do model · pk: é o identificador principal de um registro.

Por que consultar a última triagem

A busca não usa apenas a existência de uma triagem antiga apta. Subquery seleciona o resultado e a data de liberação da última concluída. A consulta exige resultado APTO e aceita liberação vazia ou já alcançada. Existe indica se o consentimento válido foi encontrado. Tudo isso se soma ao perfil, sangue compatível, preferência, conta ativa e ausência de suspensão.

Preferência e frequência

No cadastro, o Doador pode autorizar convocação; no painel, pode mudar a escolha. O helper atualiza a preferência e o registro de consentimento/revogação e audita a mudança. A frequência soma avisos de estoque e pedido dentro da janela configurada, inclusive os lidos. Cada fluxo também aplica sua proteção contra duplicatas.

Exemplo de execução

Em uma necessidade O-, a tabela define os tipos compatíveis. A busca seleciona apenas contas que passam por todas as conferências. Depois, o limite e a repetição são verificados antes de criar cada aviso. Assim, a convocação combina necessidade, compatibilidade, aptidão e autorização.

6. Central de notificações

Conteúdo do aviso

A central mostra título, mensagem, data, tipo, ação e estado lida/não lida.

Arquivo: `accounts/models.py`

```
titulo = models.CharField(max_length=120)
mensagem = models.TextField()
url_destino = models.CharField(max_length=255, blank=True, default="")
lida = models.BooleanField(default=False)
criada_em = models.DateTimeField(auto_now_add=True)
lida_em = models.DateTimeField(null=True, blank=True)
```

titulo e mensagem são o conteúdo. url_destino aponta a ação. lida indica o estado; criada_em e lida_em guardam as datas. O painel conserva o histórico em páginas de dez avisos.

CharField guarda texto curto; TextField guarda a mensagem; BooleanField guarda sim ou não; DateTimeField guarda data e hora. default=False faz o aviso começar não lido. auto_now_add=True preenche a criação automaticamente. null=True permite que a data de leitura fique vazia até a leitura acontecer.

Para ler esse código: CharField: define um campo de texto com limite de tamanho · TextField: define um campo para texto mais longo · BooleanField: define um campo de sim ou não · DateTimeField: define um campo de data e horário · max_length: define o tamanho máximo do texto.

Marcar como lida

Cada usuário marca somente seus próprios avisos.

Arquivo: `accounts/views.py`

```
notificacao = get_object_or_404(
    request.user.notificacoes, pk=id_notificacao,
)
request.user.notificacoes.filter(pk=notificacao.pk, lida=False).update(
    lida=True, lida_em=timezone.now(),
)
return redirect("accounts:dashboard")
```

request.user.notificacoes limita a busca ao dono. update marca lida e registra a data apenas se ainda não estava lida. redirect volta ao painel. O código anterior a esse trecho confere o identificador recebido.

get_object_or_404 procura o identificador dentro dos avisos do usuário atual. Se o aviso não está nesse conjunto, a ação é interrompida. O filtro lida=False faz a atualização somente na primeira leitura; timezone.now fornece o instante registrado. O aviso permanece disponível no histórico.

Para ler esse código: filter: seleciona registros pelas condições informadas · update: altera os campos dos registros selecionados · get_object_or_404: busca o registro e interrompe com 404 se ele não existir na busca permitida · request: reúne as informações do pedido enviado pelo navegador · pk: é o identificador principal de um registro.

Os botões levam a Ver estoque ou Ver pedido. Abrir o destino não marca leitura sozinho: há uma ação específica. Ler o aviso não remove a contagem de frequência.

Notificações: do botão ao banco

O formulário de leitura

O botão envia a ação e o identificador do aviso ao dashboard.

Arquivo: templates/accounts/dashboard.html

```
<form method="post">
  {% csrf_token %}
  <input type="hidden" name="acao" value="marcar_notificacao_lida">
  <input type="hidden" name="id_notificacao" value="{{ notificacao.pk }}">
  <button type="submit">Marcar como lida</button>
</form>
```

method="post" envia a mudança ao servidor. csrf_token inclui a proteção do formulário. Os campos hidden são enviados sem aparecer como entradas editáveis. acao distingue a leitura do formulário de preferência; id_notificacao indica qual aviso será tratado.

Para ler esse código: csrf_token: inclui o token de proteção do formulário · hidden: envia um dado do formulário sem exibir uma caixa de preenchimento · pk: é o identificador principal de um registro.

O processamento no servidor

A view recebe os dados, converte o identificador para número e trata valores inválidos. Depois procura o aviso dentro de request.user.notificacoes. Esse detalhe aplica a propriedade do registro: o usuário não ganha acesso ao aviso de outra conta só por conhecer seu número.

A atualização usa lida=False para agir apenas sobre avisos ainda não lidos. Ela grava lida=True e lida_em com o instante atual. Uma nova tentativa não substitui a primeira data de leitura. O redirecionamento abre o painel novamente com o estado atualizado.

O que a central apresenta

A lista é ordenada da notificação mais recente para a mais antiga, com dez por página. Mostra contador de não lidas, tipo, título, mensagem e data. A ação usa url_destino: avisos de estoque mostram Ver estoque; pedidos mostram Ver pedido. O histórico mantém os lidos e permite navegar pelas páginas anteriores.

7. Auditoria das ações do sistema

Registrar uma ação crítica

A auditoria guarda autor, ação, alvo e contexto das operações instrumentadas.

Arquivo: `accounts/pedidos.py`

```
registrar_auditoria(
    acao=AuditoriaAcaoCritica.Acao.MODERACAO, usuario=usuario, alvo=pedido, request=request,
    descricao="Publicacao institucional de pedido de sangue.",
    metadados={"evento": "PUBLICACAO_PEDIDO", "status_pedido": pedido.status},
)
```

Neste exemplo, a publicação do pedido cria um registro de auditoria. `usuario` identifica quem agiu; `alvo` indica o pedido; `metadados` acrescenta evento e estado. O helper `central` trata os metadados antes de gravar.

`acao` classifica o evento; `usuario` informa o responsável; `alvo` aponta o registro afetado; `request` fornece contexto do acesso. `descricao` apresenta o que aconteceu em texto. `metadados` guarda detalhes organizados por nome, como `evento` e `status_pedido`, para ajudar na consulta administrativa.

Para ler esse código: `request`: reúne as informações do pedido enviado pelo navegador · `metadados`: guarda detalhes adicionais do evento por nome.

Falhas de login

O sistema registra falhas e identifica repetição como suspeita.

Arquivo: `accounts/signals.py`

```
LIMITE_LOGIN_SUSPEITO = 5
JANELA_LOGIN_SUSPEITO_MINUTOS = 10
```

Cinco falhas na janela de dez minutos geram evento suspeito. O sinal do Django avisa quando o login falha e a função `auditar_login_falho` grava o evento.

O sinal é um aviso interno do Django de que a autenticação falhou. A função registra a falha e conta as recentes para o e-mail informado, ou por IP no caminho alternativo previsto. Ao atingir o limite, acrescenta o evento `LOGIN_SUSPEITO` à auditoria.

Acessos sensíveis

Foi incluído o registro das consultas a dados sensíveis nas rotas definidas.

Arquivo: `accounts/auditoria.py`

```
ROTAS_SENSIVEIS = {
    "accounts:triagem_pergunta", "accounts:triagem_resultado",
    "accounts:triagem_historico", "accounts:painel_validacao_pedidos",
    "accounts:triagem_revisao", "accounts:minhas_solicitacoes",
    "accounts:painel_pedidos_hemocentro",
}
```

Essa lista identifica as telas acompanhadas pelo middleware. Ao terminar a resposta, ele confere rota, usuário e resultado do acesso para registrar consultas sensíveis ou tentativas bloqueadas. O painel administrativo permite consultar os registros com a autorização exigida.

Middleware é uma camada que acompanha o processamento das páginas. Aqui ele observa a resposta já produzida: uma consulta autorizada a uma rota sensível pode gerar registro de acesso; uma tentativa recusada pode gerar registro de bloqueio. A auditoria guarda o evento sem copiar o formulário clínico inteiro.

Auditoria: como o evento é salvo

Registro padronizado

As funções do sistema usam um helper para gravar os eventos no mesmo formato.

Arquivo: accounts/auditoria.py

```
tipo_detectado, id_detectado = identificar_alvo(alvo)
usuario_autenticado = getattr(usuario, "is_authenticated", False)
if usuario is not None and not usuario_autenticado:
    usuario = None
return AuditoriaAcaoCritica.objects.create(
    usuario=usuario,
    acao=acao,
    resultado=resultado,
    alvo_tipo=alvo_tipo or tipo_detectado,
    alvo_id=alvo_id or id_detectado,
    descricao=descricao,
    ip=ip or obter_ip(request),
    user_agent=user_agent or obter_user_agent(request),
    metadados=limpar_metadados(metadados or {}),
)
```

identificar_alvo obtém tipo e identificador do registro afetado. A função considera se o usuário está autenticado. O create grava o evento com ação, resultado, alvo, descrição e informações do acesso. Valores passados diretamente podem complementar os detectados.

Para ler esse código: create: cria e salva um novo registro · getattr: lê uma informação do objeto e aceita um valor padrão se ela estiver ausente · objects: dá acesso às operações de busca e gravação do model · request: reúne as informações do pedido enviado pelo navegador · metadados: guarda detalhes adicionais do evento por nome.

Dados do acesso e proteção do conteúdo

IP e user_agent ajudam a identificar o contexto da ação. limpar_metadados aplica as regras de limpeza antes da gravação. O objetivo é registrar o acontecimento e informações necessárias para a análise, evitando guardar senhas e outros valores protegidos nos campos tratados pelo helper.

Eventos e consulta administrativa

Os serviços registram ações críticas nos pontos em que elas acontecem. O sinal de login registra falhas. O middleware acompanha respostas e registra consultas sensíveis ou tentativas bloqueadas conforme as rotas definidas. No admin, permissões controlam a consulta e protegem o histórico contra alteração pela interface.